

ShopAssist

Capstone Project Evaluation Report

Coding & design quality · Architecture · Completeness & gaps · Production-readiness roadmap

Prepared for the engineering team — written for developers, not a slide deck.

Scope: the **shopassist** repository (FastAPI backend + CLI simulation).

1. Executive Summary

ShopAssist is a multi-agent, RAG-backed customer support backend (~4,850 lines of Python across 29 files) built around a genuinely well-thought-out architecture: a task-decomposition step breaks each customer message into one or more sub-tasks, routes each to a specialist agent (order tracking, product recommendation, or general-purpose RAG lookup), and synthesizes a single natural-language reply from however many agents actually ran. Two entry points — a FastAPI service and a CLI simulation script — share one services/ layer, so behavior is identical regardless of how it's driven.

For a capstone project, the engineering discipline on display is well above the typical bar: defense-in-depth guardrails (prompt-injection screening, output PII/secret redaction, an actually-enforced order-ownership check closing a real IDOR vulnerability), full request observability (Langfuse tracing wired at the correct point in the pipeline — after PII masking, not before — plus correlation-ID-threaded structured logging), a model-agnostic hybrid LLM design (Ollama-local and Gemini-cloud per pipeline role, with automatic cloud-to-local failover), and a containerized deployment with real hardening (non-root user, per-service memory limits, healthchecks, and — verified directly, not just asserted — zero build-time coupling to sibling repositories).

The gap between this and a production system is narrower than most capstones, but it is real and concentrated in a few places: **there is no automated test suite or CI pipeline** — every behavior verified during development was checked manually — and several pieces of state (conversation history, rate limiting, routing caches) live in plain in-process dictionaries, which is fine for one instance but caps the system at a single replica. A small number of features are explicitly incomplete (destructive order actions have no confirmation step; two documented chat endpoints 404). None of these are hidden — the codebase's own comments and README already flag most of them, which is itself a positive signal about the team's engineering maturity.

2. Architecture Overview

Request flow (services/orchestrator.py, mirrored in README.md):

- PII masking (services/pii_masker.py) — pattern-based, applied before anything reaches an LLM.
- Sentiment classification (services/classifier_client.py) — optional, external, fails soft to 'unknown'.
- Input guardrail screen (services/guardrails.py) — blocks blatant prompt-injection, fails open on ambiguous text.
- Task decomposition (services/llm_inference.py::call_task_decomposer) — one message can dispatch to multiple agents in one turn.
- Per-sub-task agent execution — OrderTrackingAgent, ProductRecommendationAgent, or GeneralPurposeAgent (RAG); any exception falls back to EscalationAgent.
- Generative synthesis (call_generative) — turns every sub-task's structured result into one customer-facing reply, sentiment-calibrated.
- Output guardrail screen — redacts PII/secret/credit-card-shaped substrings the LLM might echo back.

LLM abstraction (services/llm_inference.py):

Every one of five pipeline roles (router/decomposer/agent-reason/agent-interpret/generative) independently resolves to a client via `_resolve_role()`: `local` (Ollama, OpenAI-compatible) or `gemini` (Google's OpenAI-compatible endpoint), selected per role by a `<ROLE>_PROVIDER` env var. A failed cloud call transparently retries once against the local model before giving up. This is a clean, extensible pattern — adding a third OpenAI-compatible provider is a contained addition, not a rewrite — with one asymmetry: `call_embeddings()` is hardcoded to Ollama regardless of role provider settings, so the design is hybrid for generation but not yet for embeddings.

RAG (services/rag.py, services/data_pipeline.py):

FAISS flat L2 index over real embeddings (Ollama's `nomic-embed-text` by default), backed by a JSONL document store. Ingests seeded customer conversations, product reviews, and policy PDFs (chunked via LangChain's `RecursiveCharacterTextSplitter`, 1000/200 char size/overlap) at startup. `top_k` is tuned per agent — 3 for general-purpose/policy questions (to cover multi-topic queries without losing context), 3 for product recommendation.

Data & deployment:

SQLAlchemy over SQLite (host-native dev) or Postgres (Docker), switched purely by whether `DATABASE_URL` is set. Order/customer operations enforce that the calling `user_id` actually owns the resource. Docker Compose brings up exactly two services — `api` and `postgres` — with Ollama and the optional sentiment classifier both expected to run externally (on the host, or via a sibling project's own compose file), never as a build-time dependency. This was verified directly: a full clean rebuild succeeds with the sibling repo removed from disk entirely.

3. Strengths

- **Multi-agent task decomposition**, not a single-intent router — one message like “where's my order and what's your return policy” is handled correctly in one turn by two agents.
- **Real security posture for an LLM-driven backend**: input prompt-injection screening, output PII/secret/credit-card (Luhn-validated) redaction, and an order-ownership check that was found and fixed to close an actual IDOR (a previous version's check silently skipped enforcement for any `user_id` that didn't match a specific prefix).
- **Real RAG**, not a keyword-match placeholder — FAISS vector search over genuine embeddings, ingesting structured product/customer data and unstructured policy PDFs alike.
- **Model-agnostic, hybrid LLM routing** with automatic cloud-to-local failover per pipeline role — a deliberately engineered capability, not an afterthought.
- **Observability done correctly, not just present**: Langfuse tracing is wired in *after* PII masking and output redaction, not before — so raw customer PII is never shipped to a third-party tracing backend. Every log line carries a request correlation ID.
- **Fail-soft discipline applied consistently** — the sentiment classifier, RAG, embeddings, and every LLM call degrade to a safe default under failure rather than crashing a request or the whole process.
- **Container hygiene most capstones skip entirely**: non-root user, per-service memory limits, healthchecks gating startup order, and zero coupling to sibling repositories at build time (independently verified, not just claimed in a comment).
- **Documentation quality is genuinely unusual** — README.md, api/README.md, and db/README.md are accurate and current, and in-code comments consistently explain *why* a decision was made, not just what the code does. This materially lowers onboarding cost and review effort.

4. Weaknesses & Gaps

- **No automated tests, no CI pipeline**. Every behavior described in this report was verified by manual curl requests and log inspection. This is the single largest gap standing between this project and production readiness, and the one most likely to bite as the codebase grows.
- **In-memory-only state** — conversation history, per-session agent state, the routing/decomposition cache, and the rate limiter are all plain process-local dictionaries. Correct for one instance; will silently misbehave (lost history, ineffective rate limiting) the moment a second replica or a restart enters the picture.
- **No confirmation step before destructive order actions**. OrderTrackingAgent's cancel/delete tools execute on a single LLM reasoning decision. Ownership is enforced (you can't touch someone else's order), but a prompt-injection success or a reasoning mistake could still cancel or permanently delete the caller's own order with no “are you sure” turn.

- **ProductRecommendationAgent's RAG-fallback path returns a hardcoded product_id/name/price** when no direct catalog tool call succeeds — only the RAG-matched description snippet and customer-history framing are real in that path. (The direct `searchItems` tool-call path is not affected — it returns genuine catalog data.)
- **Two chat endpoints are documented but not implemented** — `GET /api/v1/chat/{session_id}/history` and `DELETE /api/v1/chat/{session_id}` appear in the router's own docstring and 404 in practice.
- **Auth is a shared dummy API key** (constant-time compared, toggleable enforcement), explicitly documented as a placeholder for real per-user JWT auth — appropriate for a demo, not for production multi-tenant use.
- **Embeddings are not part of the hybrid provider design** — `call_embeddings()` always uses Ollama regardless of what the chat-role providers are set to.
- **The input guardrail only screens the current message**, not accumulated conversation history — a jailbreak attempt spread across several turns would not be caught by design (documented in README as a known, deliberate scope limit).
- **No dependency or image vulnerability scanning** (pip-audit, Dependabot, Trivy or similar) integrated anywhere in the project.
- **No DB migration tooling** (e.g. Alembic) — schema changes are raw SQL files applied only to a fresh volume, which is workable for a capstone but not for iterative production schema evolution.

5. Pros & Cons of the Overall Implementation

Pros	Cons
Sophisticated multi-agent + real RAG architecture, well beyond typical capstone scope	Zero automated tests; no CI pipeline
Security-conscious: injection screening, output redaction, IDOR closed, dummy-but-toggleable auth	In-memory state caps the system at a single replica
Observability wired correctly (post-mask, post-redaction) with correlation IDs throughout	No confirmation gate before destructive order actions
Model-agnostic hybrid LLM routing with automatic cloud→local failover	A few explicitly-known incomplete features (hardcoded fallback recommendation, 2 missing endpoints)
Consistent fail-soft design — no single downstream outage takes down a request	Embeddings not yet part of the hybrid/model-agnostic design
Container hygiene: non-root, resource limits, healthchecks, zero sibling-repo build coupling	No dependency/image vulnerability scanning or DB migration tooling
Exceptionally clear, current documentation and in-code rationale comments	Input guardrail is single-message only; not a full moderation system by design

6. Completeness Assessment by Layer

Layer	Status	Notes
API (FastAPI)	Mostly complete	2 documented endpoints unimplemented; auth/rate-limit/correlation-ID middleware all solid
Orchestration & agents	Complete for covered domains	Order tracking, product recommendation, general/RAG, escalation fallback all functional
RAG	Functionally complete	Single embedding provider (Ollama); no re-ranking or hybrid keyword+vector search

Layer	Status	Notes
Guardrails	Solid v1, explicitly scoped	Pattern-based, single-message; not a full moderation/classifier system
Observability	Complete, correctly wired	Langfuse + correlation IDs + structured logs across every layer
Data layer	Complete for demo scope	Dual SQLite/Postgres; ownership-enforced; no migration tooling
Testing	Absent	No unit, integration, or contract tests found anywhere in the repo
CI/CD	Absent	No workflow files, no automated build/lint/scan gate found
Documentation	Excellent	README.md, api/README.md, db/README.md all present, accurate, and current

7. Roadmap Toward Production Readiness

Ordered by leverage — each item is written so the team can lift much of this session's own manual verification directly into the first roadmap item, rather than starting from zero.

7.1 Testing & CI (highest priority)

- Unit tests for the cheap, pure-function-shaped pieces first: `services/guardrails.py` (injection patterns, Luhn validation, email allowlist), `services/pii_masker.py`, and `clients/ecommerce_api_client.py`'s ownership checks.
- Integration tests for `services/orchestrator.py` with a mocked `LLMInferenceService/EcommerceClient` — verifies routing/decomposition/fallback logic without needing a live Ollama or Postgres.
- A CI workflow (GitHub Actions) that at minimum lints, runs the above, and — for real end-to-end confidence — brings up docker compose and runs a scripted curl smoke suite mirroring this report's own verification steps (health, chat, guardrail block, ownership block).

7.2 Statefulness

- Move conversation history, agent state, and rate-limit counters to Redis (the codebase's own comments already name this as the intended next store) before running more than one replica.

7.3 Safety

- Add an explicit confirmation turn before `OrderTrackingAgent` executes cancel/delete.
- Replace the shared dummy API key with real per-user JWT auth once a login/identity system is in front of the gateway (`shopassist-client` already has one).

7.4 Model & retrieval quality

- Extend the hybrid provider design to embeddings (currently Ollama-only regardless of chat-role config).
- Wire `ProductRecommendationAgent`'s RAG-fallback path to real catalog data instead of the hardcoded placeholder.
- Evaluate a real moderation/toxicity classifier to replace or augment the current pattern-based input guardrail — `services/classifier_client.py` already establishes the pattern for loading such a model.

7.5 Operational hardening

- Add `pip-audit/Dependabot` and image scanning (Trivy) to CI.
- Introduce Alembic (or similar) for versioned schema migrations instead of raw SQL files applied only to fresh volumes.

8. Suggestions to Take It Forward (Prioritized)

- 1 **Stand up a test suite and CI pipeline first.** It's the one gap that makes every other improvement riskier to make; everything else on this list gets safer to build once this exists.
- 2 **Move shared state to Redis** before any discussion of scaling beyond one instance.
- 3 **Add a confirmation step for destructive order actions** — small, contained, and closes the last significant safety gap alongside the ownership fix already in place.
- 4 **Replace the dummy API key with real per-user auth** once identity is available end to end.
- 5 **Extend the hybrid design to embeddings and evaluate a real moderation classifier** — both build directly on patterns the codebase already established for chat-role provider switching and in-process model loading.

Prepared from a direct review of the shopassist repository's source, configuration, and Docker deployment — findings in this report were verified against running code, not inferred from documentation alone.